

Vibing Insecurities

Autor: Ludwig Alvarado

La Unidad Técnica de Agentes, Detectives y Operaciones (UTADEO) es una organización clandestina que se dedica al espionaje académico y a la manipulación de registros de calificaciones. Muchos estudiantes recurren a sus servicios para alterar sus notas, mientras que la organización identifica posibles clientes analizando sus inseguridades durante los exámenes. A este proceso lo denominan *vibing insecurities*.

Además, UTADEO se aprovecha del auge del *vibe coding*, donde múltiples instituciones han delegado el desarrollo de sus sistemas a modelos de lenguaje y programadores poco rigurosos. Estas vulnerabilidades permiten que el equipo de inyección SQL modifique la base de datos académica sin ser detectado.

Tú haces parte de este equipo. Tu misión es implementar un sistema que procese una serie de operaciones sobre los registros de estudiantes y sus calificaciones.

Cada estudiante tiene:

- Un nombre único
- Una lista de calificaciones

El puntaje final de un estudiante se define como el promedio de sus calificaciones.

Entrada del programa

La primera línea contiene un entero N , el número inicial de estudiantes.

Las siguientes N líneas contienen la información de cada estudiante en el formato:

```
name k score_1 score_2 ... score_k
```

Donde:

- `name` es una cadena en minúsculas
- k es la cantidad de calificaciones

Luego se proporciona un entero Q — el número de consultas.

Las siguientes Q líneas contienen una de las siguientes operaciones:

- `ADD name k score_1 ... score_k`
- `UPDATE name k score_1 ... score_k`
- `APPEND name score`
- `TOP k`
- `AVERAGE name`
- `GLOBAL_AVERAGE`

Descripción de las operaciones

A continuación se describe el comportamiento exacto de cada operación:

■ **ADD name k score_1 ... score_k**

Agrega un nuevo estudiante con nombre **name** y una lista de k calificaciones.

- Si el estudiante no existe, se inserta en el sistema con sus calificaciones.
- Si el estudiante ya existe, la operación se ignora y no se realiza ningún cambio.

■ **UPDATE name k score_1 ... score_k**

Reemplaza completamente la lista de calificaciones de un estudiante existente.

- Si el estudiante existe, su lista de calificaciones anterior se elimina y se reemplaza por la nueva.
- Si el estudiante no existe, la operación se ignora.

■ **APPEND name score**

Agrega una nueva calificación al final de la lista de un estudiante.

- Si el estudiante existe, la calificación se añade a su lista.
- Si el estudiante no existe, la operación se ignora.

■ **TOP k**

Consulta los k estudiantes con mejor desempeño según los siguientes criterios:

1. Mayor promedio de calificaciones
2. En caso de empate, mayor cantidad de calificaciones
3. En caso de empate, menor nombre en orden lexicográfico

- Se deben imprimir exactamente k estudiantes o todos los existentes si hay menos de k .
- Cada estudiante se imprime en una línea en el formato:
name average
- El promedio debe mostrarse con exactamente 2 decimales.

■ **AVERAGE name**

Consulta el promedio de un estudiante específico.

- Si el estudiante existe, se imprime su promedio con exactamente 2 decimales.
- Si el estudiante no existe, se imprime:
NA

■ **GLOBAL_AVERAGE**

Calcula el promedio global considerando todas las calificaciones de todos los estudiantes.

- Se deben sumar todas las calificaciones y dividir entre el total de calificaciones.
- El resultado se imprime con exactamente 2 decimales.
- Se garantiza que siempre existe al menos una calificación en el sistema al momento de esta consulta.

Salida del programa

Para cada consulta que genere salida, imprime el resultado en una nueva línea.

- TOP k : imprime los k mejores estudiantes, cada uno en una línea como:
name average
- AVERAGE name: imprime el promedio con 2 decimales, o NA si no existe
- GLOBAL_AVERAGE: imprime el promedio global con 2 decimales

Nota:

- ADD, UPDATE y APPEND no generan salida
- Las salidas deben imprimirse en el orden de las consultas

Restricciones

- $1 \leq N, Q \leq 10^5$
- $1 \leq k \leq 100$
- $0 \leq score \leq 10^6$
- Longitud del nombre ≤ 20

Caso de prueba de ejemplo

Prueba #1	
Entrada	Salida
2	alice 90.00
alice 3 90 80 100	bob 75.00
bob 2 70 80	bob 83.33
7	alice 90.00
TOP 2	90.00
APPEND bob 100	86.67
TOP 2	bob 83.33
AVERAGE alice	alice 50.00
GLOBAL_AVERAGE	
UPDATE alice 2 50 50	
TOP 2	

Notas

- El promedio se calcula como $\frac{\sum scores}{|scores|}$
- Todas las salidas numéricas deben imprimirse con exactamente 2 decimales
- Si una operación hace referencia a un estudiante inexistente, debe ignorarse (excepto en AVERAGE, donde se imprime NA)
- Se recomienda evitar recalcular promedios desde cero en cada consulta por eficiencia

Aspectos Importantes

1. La prueba se resolverá en **2 momentos**:
 - Desarrollo del algoritmo en papel (**3.0 puntos**).
 - Implementación en el editor de código (**2.0 puntos**).
2. Para leer los datos puede utilizar la función `split()` de la clase `str` y de ahí indexar los datos en un diccionario con llave de tipo `str` y valor de tipo `list`. Por ejemplo, para procesar la línea `alice 3 90 80 100` esto se puede guardar dentro de un diccionario como:

```
1 >>> entrada = input().split()
2 >>> diccionario = {}
3 >>> diccionario[entrada[0]] = list(map(int, entrada[1:]))
4 >>> print(diccionario)
5 {"alice": [3, 90, 80, 100]}
6
```

3. Recuerda que la función `round()` ayuda a redondear números decimales. Por ejemplo, el número `3.141592653589793` al aplicar `round(3.141592653589793, 2)` esto retorna en `3.14`
4. Este ejercicio evalúa los temas vistos hasta el **segundo corte**.
5. No es necesario usar estructuras de datos avanzadas ni importar librerías externas, **ante la duda pregunte al profesor o monitore, de lo contrario, su nota se verá afectada**.
6. No es necesario validar errores de entrada.
7. La única conversión de tipo de datos debe ser al momento de la lectura de los mismos. No es necesario volver a convertir el tipo de dato de las variables.
8. El formato de salida debe coincidir exactamente con el especificado. Cualquier diferencia será considerada incorrecta.